Production Deployment Topology - Docker & On-Premise Server

Deskripsi

Dokumentasi ini menjelaskan arsitektur production deployment untuk Document Management System menggunakan Docker containers pada server on-premise dengan konfigurasi high availability dan security best practices.

---

1. Production Architecture Diagram

```
graph TB
    subgraph "Internet"
        Users[👥 Users/Clients]
        Internet[🌐 Internet]
    end

    subgraph "DMZ Zone"
        Firewall[🛡️ Firewall
Port 80, 443]
        LoadBalancer[⚖️ Load Balancer
HAProxy/Nginx
Port 80, 443]
    end

    subgraph "Production Server - On Premise"
        subgraph "Docker Host"
            subgraph "Reverse Proxy Container"
                NginxProd[🔄 Nginx
Port 80, 443
SSL/TLS]
            end

            subgraph "Application Containers"
                Frontend1[🖥️ Frontend Container 1
Nginx + React Build
Port 3000]
                Frontend2[🖥️ Frontend Container 2
Nginx + React Build
Port 3001]
                Backend1[⚙️ Backend Container 1
Node.js + Express
Port 5001]
                Backend2[⚙️ Backend Container 2
Node.js + Express
Port 5002]
            end

            subgraph "Database Container"
                PostgreSQL[(🗄️ PostgreSQL
Port 5432
+ Volume Mount)]
            end

            subgraph "Supporting Services"
                Redis[💾 Redis Cache
Port 6379]
```

```
                FileStorage[📁 File Storage
Volume Mount
/uploads]
            end
        end

        subgraph "Monitoring & Logging"
            Prometheus[📊 Prometheus
Port 9090]
            Grafana[📈 Grafana
Port 3030]
            Loki[📝 Loki Logs
Port 3100]
        end
    end

    subgraph "Backup Server"
        BackupStorage[💾 Backup Storage
NAS/External]
    end

    Users --> Internet
    Internet --> Firewall
    Firewall --> LoadBalancer
    LoadBalancer --> NginxProd

    NginxProd --> Frontend1
    NginxProd --> Frontend2
    Frontend1 -.API Calls.-> NginxProd
    Frontend2 -.API Calls.-> NginxProd

    NginxProd --> Backend1
    NginxProd --> Backend2

    Backend1 --> PostgreSQL
    Backend2 --> PostgreSQL
    Backend1 --> Redis
    Backend2 --> Redis
    Backend1 --> FileStorage
    Backend2 --> FileStorage

    PostgreSQL -.Backup.-> BackupStorage
    FileStorage -.Backup.-> BackupStorage

    NginxProd -.Metrics.-> Prometheus
    Backend1 -.Metrics.-> Prometheus
    Backend2 -.Metrics.-> Prometheus
    PostgreSQL -.Metrics.-> Prometheus

    Prometheus --> Grafana
    Backend1 -.Logs.-> Loki
    Backend2 -.Logs.-> Loki
    NginxProd -.Logs.-> Loki
```

---

2. Docker Compose Stack Architecture

```
graph LR
    subgraph "docker-compose.yml"
```

```
        subgraph "Web Tier"
            N[nginx-proxy
:80, :443]
            F1[frontend-1
:3000]
            F2[frontend-2
:3001]
        end

        subgraph "Application Tier"
            B1[backend-1
:5001]
            B2[backend-2
:5002]
        end

        subgraph "Data Tier"
            DB[(postgres
:5432)]
            R[redis
:6379]
            V1[volume-db]
            V2[volume-uploads]
        end

        subgraph "Monitoring Tier"
            P[prometheus
:9090]
            G[grafana
:3030]
            L[loki
:3100]
        end
    end

    N --> F1
    N --> F2
    N --> B1
    N --> B2

    B1 --> DB
    B2 --> DB
    B1 --> R
    B2 --> R

    DB --- V1
    B1 --- V2
    B2 --- V2

    B1 -.-> P
    B2 -.-> P
    DB -.-> P
    P --> G

    B1 -.-> L
    B2 -.-> L
    N -.-> L
```

3. Network Flow - Production Request Sequence

```
sequenceDiagram
    participant U as User Browser
    participant F as Firewall
    participant LB as Load Balancer
    participant N as Nginx Proxy
    participant FE as Frontend Container
    participant BE as Backend Container
    participant R as Redis Cache
    participant DB as PostgreSQL
    participant FS as File Storage

    Note over U,FS: Initial Page Load
    U->>F: HTTPS Request (port 443)
    F->>LB: Forward (SSL Termination)
    LB->>N: HTTP/2 Request
    N->>FE: Proxy to Frontend
    FE-->>N: React App (HTML/JS/CSS)
    N-->>LB: Response
    LB-->>F: Response
    F-->>U: Encrypted Response

    Note over U,FS: User Login Flow
    U->>F: POST /api/auth/login
    F->>LB: Forward
    LB->>N: Forward
    N->>BE: Load Balance to Backend
    BE->>R: Check Session Cache
    alt Cache Hit
        R-->>BE: Session Data
    else Cache Miss
        BE->>DB: Query User
        DB-->>BE: User Data
        BE->>R: Store Session
    end
    BE-->>N: JWT Token
    N-->>LB: Response
    LB-->>F: Response
    F-->>U: Set Cookie + Token

    Note over U,FS: Document Upload Flow
    U->>F: POST /api/documents (with file)
    F->>LB: Forward
    LB->>N: Forward
    N->>BE: Route to Backend
    BE->>R: Validate Session
    R-->>BE: Session Valid
    BE->>FS: Save File to Volume
    FS-->>BE: File Path
    BE->>DB: Insert Document Record
    DB-->>BE: Document ID
    BE-->>N: Success Response
    N-->>LB: Response
    LB-->>F: Response
    F-->>U: Upload Complete
```

---

4. Container & Port Mapping

```
graph TB
    subgraph "External Access"
        Port80[Port 80
HTTP]
        Port443[Port 443
HTTPS]
    end

    subgraph "Docker Network: app-network"
        NginxC[nginx-proxy
Internal: 80, 443
External: 80, 443]

        Front1[frontend-1
Internal: 80
External: 3000]
        Front2[frontend-2
Internal: 80
External: 3001]

        Back1[backend-1
Internal: 5001
External: 5001]
        Back2[backend-2
Internal: 5002
External: 5002]

        Postgres[postgres
Internal: 5432
External: 5432*]

        RedisC[redis
Internal: 6379
External: 6379*]

        PromC[prometheus
Internal: 9090
External: 9090*]

        GrafanaC[grafana
Internal: 3000
External: 3030]

        LokiC[loki
Internal: 3100
External: 3100*]
    end

    Port80 --> NginxC
    Port443 --> NginxC

    NginxC -.Internal Network.-> Front1
    NginxC -.Internal Network.-> Front2
    NginxC -.Internal Network.-> Back1
    NginxC -.Internal Network.-> Back2

    Back1 -.-> Postgres
    Back2 -.-> Postgres
    Back1 -.-> RedisC
    Back2 -.-> RedisC
```

```
    Note1[* Only exposed to localhost
for admin access]
```

---

5. Production Technology Stack

```
mindmap
  root((Production Stack))
    Infrastructure
      Physical Server
        CPU: 8+ cores
        RAM: 16GB+
        Storage: 500GB+ SSD
      Docker Engine
        Docker Compose v2
        Docker Networks
        Docker Volumes
      Operating System
        Ubuntu 22.04 LTS
        RHEL 8+
        Debian 11+

    Frontend Layer
      Container Technology
        Docker Multi-stage Build
        Nginx Alpine Image
      Application
        React 18
        TypeScript
        Vite Build
      Replication
        2+ instances
        Round-robin LB

    Backend Layer
      Container Technology
        Node 20 Alpine
        Multi-stage Build
      Framework
        Express.js
        JWT Auth
        Sequelize ORM
      Scaling
        Horizontal Scaling
        Stateless Design
        Session in Redis

    Data Layer
      PostgreSQL 15
        Persistent Volume
        Connection Pooling
        Automated Backup
      Redis 7
        Session Store
        Cache Layer
        Pub/Sub
      File Storage
        Docker Volume
        NFS Mount
```

```
        Regular Backup

    Network Layer
      Reverse Proxy
        Nginx
        SSL/TLS
        HTTP/2
      Load Balancer
        HAProxy
        Health Checks
        Sticky Sessions
      Firewall
        iptables
        fail2ban
        Port Restriction

    Monitoring
      Metrics
        Prometheus
        Node Exporter
        Postgres Exporter
      Visualization
        Grafana Dashboards
        Alert Manager
        Uptime Monitors
      Logging
        Loki
        Promtail
        Log Aggregation

    Security
      SSL/TLS
        Let's Encrypt
        Auto-renewal
        HSTS
      Authentication
        JWT Tokens
        Password Hashing
        Rate Limiting
      Container Security
        Non-root User
        Read-only FS
        Security Scanning
      Network Security
        Private Networks
        Firewall Rules
        VPN Access
```

---

6. High Availability & Disaster Recovery

```
graph TD
    subgraph "Primary Server"
        P1[Active System
All Services Running]
    end

    subgraph "Monitoring"
        HM[Health Monitor
```

```
Every 30s]
    end

    subgraph "Backup Strategy"
        DB_Backup[Database Backup
Daily @ 2AM]
        File_Backup[File Storage Backup
Daily @ 3AM]
        Config_Backup[Config Backup
Weekly]
    end

    subgraph "Backup Storage"
        Local[Local NAS
7 days retention]
        Remote[Remote Backup
30 days retention]
    end

    subgraph "Recovery Scenarios"
        Container_Fail{Container Failure?}
        Server_Fail{Server Failure?}
        Data_Loss{Data Loss?}
    end

    HM -->|Check| P1
    P1 -->|Healthy| HM

    P1 --> DB_Backup
    P1 --> File_Backup
    P1 --> Config_Backup

    DB_Backup --> Local
    File_Backup --> Local
    Config_Backup --> Local

    Local -.Replicate.-> Remote

    P1 -.-> Container_Fail
    Container_Fail -->|Yes| Auto_Restart[Docker Auto-restart
restart: unless-stopped]
    Auto_Restart --> P1

    P1 -.-> Server_Fail
    Server_Fail -->|Yes| Manual_Restore[Manual Restore
from Backup]
    Remote --> Manual_Restore

    P1 -.-> Data_Loss
    Data_Loss -->|Yes| Point_in_Time[Point-in-Time Recovery
from Daily Backup]
    Remote --> Point_in_Time
```

---

7. Deployment Components

Component	Container Name	Image	Replicas	Port Mapping	Volume Mount	Purpose

Component	Container Name	Image	Replicas	Port Mapping	Volume Mount	Purpose
Reverse Proxy	nginx-proxy	nginx:alpine	1	80:80, 443:443	./nginx/ssl:/etc/nginx/ssl	SSL termination, routing
Frontend	frontend-1, frontend-2	documan-frontend:latest	2	3000:80, 3001:80	-	Serve React SPA
Backend API	backend-1, backend-2	documan-backend:latest	2	5001:5001, 5002:5002	./uploads:/app/uploads	REST API, business logic
Database	postgres	postgres:15-alpine	1	5432:5432	pgdata:/var/lib/postgresql/data	Persistent data storage
Cache	redis	redis:7-alpine	1	6379:6379	redis-data:/data	Session store, caching
Monitoring	prometheus	prom/prometheus	1	9090:9090	./prometheus:/etc/prometheus	Metrics collection
Dashboard	grafana	grafana/grafana	1	3030:3000	grafana-data:/var/lib/grafana	Metrics visualization
Logging	loki	grafana/loki	1	3100:3100	-	Log aggregation

8. Production Deployment Configuration

Server Requirements

Minimum Specifications:

- **CPU:** 4 cores (8 cores recommended)
- **RAM:** 8GB (16GB recommended)
- **Storage:** 250GB SSD (500GB recommended)
- **Network:** 100Mbps (1Gbps recommended)
- **OS:** Ubuntu 22.04 LTS / RHEL 8+ / Debian 11+

Docker Compose Configuration

File: `docker-compose.production.yml`

Key Features:

- Multi-container orchestration
- Health checks for all services
- Automatic restart policies
- Resource limits (CPU/Memory)
- Private network isolation
- Named volumes for persistence
- Environment variable management
- Secret management

Network Configuration

Docker Networks:

- `frontend-network` - Frontend ↔ Nginx
- `backend-network` - Backend ↔ Database/Redis
- `monitoring-network` - All services ↔ Monitoring stack

Firewall Rules (iptables):

```
# Allow HTTP/HTTPS only
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
iptables -A INPUT -p tcp --dport 443 -j ACCEPT

# Allow SSH (from specific IPs)
iptables -A INPUT -p tcp --dport 22 -s <ADMIN_IP> -j ACCEPT

# Block all other incoming
iptables -A INPUT -j DROP
```

SSL/TLS Configuration

Certificate Management:

- **Provider:** Let's Encrypt (free)
- **Auto-renewal:** Certbot with cron job
- **Protocols:** TLS 1.2, TLS 1.3 only
- **Ciphers:** Strong ciphers only (A+ rating)

Environment Variables

Production `.env` file:

```
# Application
NODE_ENV=production
APP_PORT=5001
APP_URL=https://documan.example.com

# Database
DB_HOST=postgres
DB_PORT=5432
DB_NAME=documan_production
DB_USER=documan_user
DB_PASSWORD=<STRONG_PASSWORD>
DB_POOL_MAX=20

# Redis
REDIS_HOST=redis
REDIS_PORT=6379
REDIS_PASSWORD=<REDIS_PASSWORD>

# Security
JWT_SECRET=<RANDOM_256BIT_SECRET>
SESSION_SECRET=<RANDOM_256BIT_SECRET>

# File Upload
UPLOAD_DIR=/app/uploads
MAX_FILE_SIZE=104857600

# Monitoring
```

```
PROMETHEUS_ENABLED=true
LOKI_ENABLED=true
```

---

9. Access & Security Configuration

Production Access

Public URL: `https://documan.example.com`

Admin Access:

- Grafana: `https://documan.example.com/grafana`
- Prometheus: SSH tunnel only (not public)

Database Access:

- Direct: Localhost only (SSH tunnel required)
- Backup: Automated scripts

Security Layers

1. **Network Security**
    - Firewall (only port 80, 443, 22 allowed)
    - fail2ban (brute force protection)
    - VPN for admin access
2. **Application Security**
    - JWT authentication
    - bcrypt password hashing
    - Rate limiting (100 req/min per IP)
    - CORS restrictions
    - SQL injection protection (ORM)
    - XSS protection (CSP headers)
3. **Container Security**
    - Non-root users in containers
    - Read-only root filesystem
    - Security scanning (Trivy)
    - Minimal base images (Alpine)
    - No privileged containers
4. **Data Security**
    - Encrypted database connections
    - Encrypted backups
    - Regular security updates
    - GDPR compliance

---

10. Monitoring & Alerting

Metrics Collected

- **System Metrics:** CPU, RAM, Disk, Network
- **Application Metrics:** Request rate, response time, error rate

- **Database Metrics:** Connections, query time, cache hit rate
- **Container Metrics:** Resource usage, restart count

Grafana Dashboards

1. **System Overview:** Server health, resource usage
2. **Application Performance:** API response times, throughput
3. **Database Performance:** Query performance, connections
4. **Business Metrics:** Active users, document uploads

Alert Rules

- CPU usage > 80% for 5 minutes
- Memory usage > 90%
- Disk space < 10%
- Container restart detected
- HTTP 5xx errors > 10 per minute
- Database connection pool exhausted

---

11. Backup & Recovery Strategy

Backup Schedule

Data Type	Frequency	Retention	Method
Database	Daily 2AM	30 days	pg_dump + compression
File Storage	Daily 3AM	30 days	rsync to NAS
Configuration	Weekly	90 days	Git + tar.gz
Docker Volumes	Weekly	30 days	Docker volume backup

Recovery Procedures

Database Recovery:

```
# Restore from backup
docker exec -i postgres psql -U documan_user -d postgres < backup.sql

# Point-in-time recovery
docker exec postgres pg_restore -d documan_production /backup/latest.dump
```

File Storage Recovery:

```
# Restore uploads
rsync -avz /backup/uploads/ /var/lib/docker/volumes/uploads/_data/
```

Full System Recovery:

```
# 1. Install Docker & Docker Compose
# 2. Clone repository
git clone <repo-url>
cd document-management-system
```

```
# 3. Restore configuration
cp /backup/config/.env .env

# 4. Deploy containers
docker-compose -f docker-compose.production.yml up -d

# 5. Restore database
docker exec -i postgres psql -U documan_user < /backup/db/latest.sql

# 6. Restore files
rsync -avz /backup/uploads/ ./uploads/
```

---

12. Maintenance Procedures

Regular Maintenance Tasks

Daily:

- Monitor system health (automated)
- Check backup completion
- Review error logs

Weekly:

- Update Docker images (security patches)
- Review Grafana dashboards
- Clean old logs (> 7 days)

Monthly:

- Security audit
- Performance review
- Capacity planning review
- Test disaster recovery

Scaling Procedures

Vertical Scaling (More Resources):

```
# Update docker-compose.yml
services:
  backend-1:
    deploy:
      resources:
        limits:
          cpus: '2'
          memory: 4G
```

Horizontal Scaling (More Instances):

```
# Scale backend containers
docker-compose -f docker-compose.production.yml up -d --scale backend=4

# Scale frontend containers
docker-compose -f docker-compose.production.yml up -d --scale frontend=3
```

---

13. Deployment Checklist

Pre-Deployment

- [ ] Server provisioned with required specs
- [ ] Docker & Docker Compose installed
- [ ] Domain name configured (A record)
- [ ] SSL certificate obtained
- [ ] Firewall rules configured
- [ ] Backup storage configured
- [ ] Monitoring tools installed
- [ ] `.env` file configured with production values
- [ ] Database initialized
- [ ] Security hardening completed

Deployment

- [ ] Clone repository to server
- [ ] Build Docker images
- [ ] Run database migrations
- [ ] Deploy containers with docker-compose
- [ ] Verify all containers running
- [ ] Test frontend access
- [ ] Test backend API
- [ ] Test database connectivity
- [ ] Verify SSL certificate
- [ ] Test file upload/download

Post-Deployment

- [ ] Configure automated backups
- [ ] Set up monitoring alerts
- [ ] Document admin credentials (securely)
- [ ] Test disaster recovery procedure
- [ ] Monitor logs for 24 hours
- [ ] Performance baseline established
- [ ] User acceptance testing
- [ ] Handover documentation complete

---

14. Comparison: Development vs Production

Aspect	Current Setup (Dev)	Production Setup (Docker)
Deployment	Manual process start	Docker Compose orchestration
Scaling	Single instance	Multi-container, horizontal scaling
Availability	Single point of failure	High availability with replicas
Security	Basic (Cloudflare tunnel)	Multi-layer (Firewall, SSL, Network isolation)
Monitoring	Manual checking	Automated (Prometheus + Grafana)
Backup	Manual	Automated daily backups

Aspect	Current Setup (Dev)	Production Setup (Docker)
Recovery	Manual restart	Auto-restart + disaster recovery
SSL/TLS	Cloudflare managed	Let's Encrypt with auto-renewal
Database	Local PostgreSQL	Containerized with volume persistence
Caching	None	Redis for sessions and caching
Logging	File-based	Centralized (Loki)
Cost	Free (Cloudflare Quick Tunnel)	Server hosting cost
Performance	Development optimized	Production optimized
Access	Temporary URL	Permanent domain

Kesimpulan

Implementasi production dengan Docker dan server on-premise memberikan:

✅ **High Availability** - Multiple container replicas dengan load balancing
✅ **Scalability** - Mudah scale horizontal dengan tambah container
✅ **Security** - Multi-layer security dari firewall hingga application
✅ **Monitoring** - Real-time metrics dan alerting
✅ **Disaster Recovery** - Automated backup dan recovery procedures
✅ **Maintainability** - Container orchestration dengan Docker Compose
✅ **Performance** - Resource optimization dan caching layer
✅ **Cost Control** - One-time server investment, no recurring cloud costs

Estimated Setup Time: 2-3 hari untuk full production deployment
Estimated Server Cost: $100-300/month (dedicated server) atau one-time hardware investment